gRPConf India 2026 is on **Nov 18th at Bengaluru!** - **Register now** . Checkout **Schedule**

Asynchronous-API tutorial

This tutorial shows you how to write a simple server and client in C++ using gRPC's asynchronous/non-blocking APIs. It assumes you are already familiar with writing simple synchronous gRPC code, as described in Basics tutorial. The example used in this tutorial follows from the basic Greeter example used in the quick start. You'll find it along with installation instructions in grpc/examples/cpp/helloworld.

Overview

gRPC uses the CompletionQueue API for asynchronous operations. The basic work flow is as follows:

- bind a `CompletionQueue` to an RPC call
- do something like a read or write, present with a unique `void*` tag
- call `CompletionQueue::Next` to wait for operations to complete. If a tag appears, it indicates that the corresponding operation is complete.

Async client

To use an asynchronous client to call a remote method, you first create a channel and stub, just as you do in a synchronous client. Once you have your stub, you do the following to make an asynchronous call:

- Initiate the RPC and create a handle for it. Bind the RPC to a `CompletionQueue` .

```
CompletionQueue cq;
std::unique_ptr<ClientAsyncResponseReader<HelloReply> > rpc(
    stub_->AsyncSayHello(&context, request, &cq));
```

- Ask for the reply and final status, with a unique tag

```
Status status;
rpc->Finish(&reply, &status, (void*)1);
```

- Wait for the completion queue to return the next tag. The reply and status are ready once the tag passed into the corresponding `Finish()` call is returned.

```
void* got_tag;
bool ok = false;
cq.Next(&got_tag, &ok);
if (ok && got_tag == (void*)1) {
  // check reply and status
}
```

You can see the complete client example in greeter_async_client.cc.

Async server

The server implementation requests an RPC call with a tag and then waits for the completion queue to return the tag. The basic flow for handling an RPC asynchronously is:

- Build a server exporting the async service

```
helloworld::Greeter::AsyncService service;
ServerBuilder builder;
builder.AddListeningPort("0.0.0.0:50051", InsecureServerCredentials());
builder.RegisterService(&service);
auto cq = builder.AddCompletionQueue();
auto server = builder.BuildAndStart();
```

- Request one RPC, providing a unique tag

```
ServerContext context;
HelloRequest request;
ServerAsyncResponseWriter<HelloReply> responder;
service.RequestSayHello(&context, &request, &responder, &cq, &cq, (void*)1);
```

- Wait for the completion queue to return the tag. The context, request and responder are ready once the tag is retrieved.

```
HelloReply reply;
Status status;
void* got_tag;
bool ok = false;
cq.Next(&got_tag, &ok);
if (ok && got_tag == (void*)1) {
  // set reply and status
  responder.Finish(reply, status, (void*)2);
}
```

- Wait for the completion queue to return the tag. The RPC is finished when the tag is back.

```
void* got_tag;
bool ok = false;
cq.Next(&got_tag, &ok);
if (ok && got_tag == (void*)2) {
  // clean up
}
```

This basic flow, however, doesn't take into account the server handling multiple requests concurrently. To deal with this, our complete async server example uses a `CallData` object to maintain the state of each RPC, and uses the address of this object as the unique tag for the call.

```
class CallData {
public:
  // Take in the "service" instance (in this case representing an asynchronous
  // server) and the completion queue "cq" used for asynchronous communication
  // with the gRPC runtime.
  CallData(Greeter::AsyncService* service, ServerCompletionQueue* cq)
      : service_(service), cq_(cq), responder_(&ctx_), status_(CREATE) {
    // Invoke the serving logic right away.
    Proceed();
  }

  void Proceed() {
    if (status_ == CREATE) {
      // As part of the initial CREATE state, we *request* that the system
      // start processing SayHello requests. In this request, "this" acts are
      // the tag uniquely identifying the request (so that different CallData
      // instances can serve different requests concurrently), in this case
      // the memory address of this CallData instance.
      service_->RequestSayHello(&ctx_, &request_, &responder_, cq_, cq_,
                                this);
      // Make this instance progress to the PROCESS state.
      status_ = PROCESS;
    } else if (status_ == PROCESS) {
      // Spawn a new CallData instance to serve new clients while we process
      // the one for this CallData. The instance will deallocate itself as
      // part of its FINISH state.
      new CallData(service_, cq_);

      // The actual processing.
      std::string prefix("Hello ");
      reply_.set_message(prefix + request_.name());

      // And we are done! Let the gRPC runtime know we've finished, using the
      // memory address of this instance as the uniquely identifying tag for
      // the event.
      responder_.Finish(reply_, Status::OK, this);
      status_ = FINISH;
    } else {
      GPR_ASSERT(status_ == FINISH);
      // Once in the FINISH state, deallocate ourselves (CallData).
      delete this;
    }
  }
}
```

For simplicity the server only uses one completion queue for all events, and runs a main loop in `HandleRpcs` to query the queue:

```
void HandleRpcs() {
  // Spawn a new CallData instance to serve new clients.
  new CallData(&service_, cq_.get());
  void* tag;  // uniquely identifies a request.
  bool ok;
  while (true) {
    // Block waiting to read the next event from the completion queue. The
    // event is uniquely identified by its tag, which in this case is the
    // memory address of a CallData instance.
    cq_->Next(&tag, &ok);
    GPR_ASSERT(ok);
    static_cast<CallData*>(tag)->Proceed();
  }
}
```

Shutting Down the Server

We've been using a completion queue to get the async notifications. Care must be taken to shut it down *after* the server has also been shut down.

Remember we got our completion queue instance `cq_` in `ServerImpl::Run()` by running `cq_ = builder.AddCompletionQueue()`. Looking at `ServerBuilder::AddCompletionQueue`'s documentation we see that

> ... Caller is required to shutdown the server prior to shutting down the returned completion queue.

Refer to `ServerBuilder::AddCompletionQueue`'s full docstring for more details. What this means in our example is that `ServerImpl's` destructor looks like:

```
~ServerImpl() {
  server_->Shutdown();
  // Always shutdown the completion queue after the server.
  cq_->Shutdown();
}
```

You can see our complete server example in greeter_async_server.cc.

Last modified February 17, 2022: rename RegisterAsyncService to RegisterService (#891) (8f2378a)

View page source | Edit this page | Create child page | Create documentation issue | Create project issue